

Projet SDP – Apprentissage de modèles MR-Sort

Côme-Alexis Puech

Hugo Vigna

Salomé Fonvielle

Octobre 2026

Ce rapport présente notre programme pour le Jalon 1, qui apprend les paramètres d'un modèle MR-Sort à partir d'un learning set en résolvant un MILP avec Gurobi. La première partie construit ce MILP, la seconde décrit son implémentation et sa validation. Les réponses détaillées aux questions préliminaires i à iv, sur l'exemple de l'admission d'étudiants, sont en annexe.

Formulation du MILP

Cette partie reprend le raisonnement des questions iv à viii bis du sujet.

Maximiser une marge

Avec deux catégories, on note A^* et R^* les objets acceptés et rejetés du learning set, et $s(x) = \sum_{i: x_i \geq b_i} w_i$ le score de x , c'est-à-dire le poids des critères sur lesquels il atteint la frontière b . Le modèle classe bien x si $s(x) \geq \lambda$ pour $x \in A^*$ et $s(x) < \lambda$ pour $x \in R^*$. On cherche donc w et λ qui maximisent une marge α , le plus petit écart entre un score et λ :

$$\max \alpha \quad \text{s.c.} \quad s(x) \geq \lambda + \alpha \quad (x \in A^*), \quad s(x) \leq \lambda - \alpha \quad (x \in R^*), \quad \sum_{i \in \mathcal{N}} w_i = 1.$$

Si $\alpha^* > 0$, tous les objets sont bien classés, sinon aucun modèle ne colle parfaitement aux données. Quand b est connu, $s(x)$ est linéaire en w et c'est un simple PL. Pour apprendre aussi b , il faut exprimer $s(x)$ de façon linéaire en fonction de b .

Linéarisation pour apprendre les frontières

Variables $\delta_i(x)$. Pour savoir si x valide le critère i , on introduit une variable binaire $\delta_i(x) = \mathbb{1}_{\{x_i \geq b_i\}}$. Comme b_i est une variable, cette indicatrice n'est pas linéaire. On l'impose avec deux contraintes, une par sens de l'équivalence ($\forall i \in \mathcal{N}, \forall x$) :

$$\begin{aligned} x_i - b_i &\geq M(\delta_i(x) - 1) & (1) \quad \delta_i(x) = 1 \Rightarrow x_i \geq b_i \\ x_i - b_i &\leq M\delta_i(x) - \varepsilon & (2) \quad \delta_i(x) = 0 \Rightarrow x_i < b_i \end{aligned}$$

On normalise les données dans $[0, 1]$ sur chaque critère, ce qui ne change pas les affectations. On a alors $-1 \leq x_i - b_i \leq 1$, donc avec $M = 2$ la contrainte qui ne correspond pas à la valeur de $\delta_i(x)$ est toujours vérifiée. Comme le solveur ne gère pas les inégalités strictes, on écrit $x_i < b_i$ sous la forme $x_i \leq b_i - \varepsilon$, avec ε plus petit que l'écart entre deux valeurs voisines des données.

Contrainte (1)		Contrainte (2)	Conclusion
$\delta_i(x) = 1$	$x_i - b_i \geq 0$	$x_i - b_i \leq M - \varepsilon$ (toujours vraie)	$x_i \geq b_i$
$\delta_i(x) = 0$	$x_i - b_i \geq -M$ (toujours vraie)	$x_i - b_i \leq -\varepsilon$	$x_i < b_i$

Variables $w_i(x)$. Le score s'écrit alors $s(x) = \sum_{i \in \mathcal{N}} w_i \delta_i(x)$, mais $w_i \delta_i(x)$ est un produit de deux variables. On le remplace par une variable $w_i(x) \geq 0$ et les contraintes

$$w_i \geq w_i(x) \geq 0 \quad (3)$$

$$\delta_i(x) \geq w_i(x) \geq \delta_i(x) + w_i - 1 \quad (4)$$

qui forcent $w_i(x) = w_i \delta_i(x)$ (les autres bornes sont toujours vraies car $0 \leq w_i \leq 1$) :

	Borne inférieure	Borne supérieure	Conclusion
$\delta_i(x) = 1$	$w_i(x) \geq w_i$ (4)	$w_i(x) \leq w_i$ (3)	$w_i(x) = w_i$
$\delta_i(x) = 0$	$w_i(x) \geq 0$ (3)	$w_i(x) \leq 0$ (4)	$w_i(x) = 0$

Le score $s(x) = \sum_{i \in \mathcal{N}} w_i(x)$ est donc linéaire, et on n'a plus besoin de connaître b à l'avance.

Le MILP à deux catégories

On reprend le programme de la marge avec ce score et on ajoute les contraintes (1) à (4) ($M = 2$, ε petit).

Variables.

- du modèle : $w_i \geq 0$, $b_i \in [0, 1 + \varepsilon]$ pour $i \in \mathcal{N}$ (au-delà de 1, aucun objet n'atteint la frontière), et $\lambda \in [0, 1]$;
- de la marge : α ;
- pour chaque objet x et chaque critère i : $\delta_i(x) \in \{0, 1\}$ et $w_i(x) \geq 0$.

Programme.

$$\begin{aligned}
 \max \quad & \alpha \\
 \text{s.c.} \quad & \sum_{i \in \mathcal{N}} w_i(x) \geq \lambda + \alpha & \forall x \in A^* & \text{(acceptés)} \\
 & \sum_{i \in \mathcal{N}} w_i(x) \leq \lambda - \alpha & \forall x \in R^* & \text{(rejetés)} \\
 & x_i - b_i \geq M(\delta_i(x) - 1) & \forall i, \forall x & (1) \\
 & x_i - b_i \leq M\delta_i(x) - \varepsilon & \forall i, \forall x & (2) \\
 & w_i \geq w_i(x) \geq 0 & \forall i, \forall x & (3) \\
 & \delta_i(x) \geq w_i(x) \geq \delta_i(x) + w_i - 1 & \forall i, \forall x & (4) \\
 & \sum_{i \in \mathcal{N}} w_i = 1
 \end{aligned}$$

Les $\delta_i(x)$ sont binaires, c'est donc un programme linéaire en nombres entiers (MILP) avec $n|A^* \cup R^*|$ variables binaires. Comme plus haut, si $\alpha^* > 0$ le modèle appris restitue tout le learning set. Si les données ne sont pas séparables ($\alpha^* < 0$), maximiser α réduit la pire erreur, ce qui n'est pas la même chose que maximiser le nombre d'objets bien classés. Ce cas ne se produit pas dans nos tests, puisque les learning sets sont générés par un modèle MR-Sort.

Généralisation à p catégories

Les catégories vont de $\mathcal{C}^1$ (la pire) à $\mathcal{C}^p$ (la meilleure) et on cherche $p - 1$ frontières $b^1, \dots, b^{p-1}, b^h$ séparant $\mathcal{C}^h$ et $\mathcal{C}^{h+1}$. Les poids et λ sont les mêmes pour toutes les frontières, seules les valeurs b_i^h changent. Un objet est dans $\mathcal{C}^h$ s'il franchit b^{h-1} mais pas b^h . On reprend donc les variables $\delta_i^h(x)$, $w_i^h(x)$ et les contraintes (1) à (4) pour chaque frontière h , avec le score $s_h(x) = \sum_{i \in \mathcal{N}} w_i^h(x)$.

Programme.

$$\begin{aligned}
 \max \quad & \alpha \\
 \text{s.c.} \quad & s_{h-1}(x) = \sum_{i \in \mathcal{N}} w_i^{h-1}(x) \geq \lambda + \alpha & \forall x \in \mathcal{C}^h, h \geq 2 & \text{(franchit } b^{h-1}) \\
 & s_h(x) = \sum_{i \in \mathcal{N}} w_i^h(x) \leq \lambda - \alpha & \forall x \in \mathcal{C}^h, h \leq p - 1 & \text{(ne franchit pas } b^h) \\
 & (1) \text{ à } (4) \text{ avec } b^h, \delta_i^h(x), w_i^h(x) & \forall i, \forall x, \forall h \\
 & b_i^{h-1} \leq b_i^h & \forall i, 2 \leq h \leq p - 1 & \text{(frontières ordonnées)} \\
 & \sum_{i \in \mathcal{N}} w_i = 1
 \end{aligned}$$

Pour $p = 2$ on retrouve le MILP précédent. Pour un objet de $\mathcal{C}^h$, seules b^{h-1} et b^h apparaissent dans ses contraintes, donc dans le code on ne crée les variables que pour ces deux frontières (au plus $2n$ binaires par objet). C'est ce programme que l'on résout avec Gurobi dans la partie suivante.

Jalon 1 : implémentation et validation

Le programme

Le MILP (cas à p catégories) est codé en Python avec `gurobipy`, dans `src/mrsort.py`. Le programme lit un learning set au format CSV (une ligne par objet : les n performances puis la classe), en déduit n et p , normalise les données, résout le MILP et affiche w , λ , les frontières (dans l'échelle d'origine), α^* et le taux de bonne classification sur le learning set.

Modèle appris sur Dataset0

Dataset0 contient 60 objets, 5 critères à maximiser et 3 catégories. On obtient en moins d'une seconde un modèle qui classe correctement les 60 objets ($\alpha^* = 0.071$) :

	crit1	crit2	crit3	crit4	crit5	
Poids w_i	1/7	2/7	1/7	1/7	2/7	$\lambda = 0.643 \approx 4.5/7$
Frontière b^1 (catégorie 1 / 2)	34.5	35	33	34.5	33	
Frontière b^2 (catégorie 2 / 3)	41.5	49	33	43.5	49	

Les critères 2 et 5 comptent double. Pour franchir une frontière, il faut l'atteindre sur des critères qui totalisent au moins 5 points sur 7 (par exemple crit2, crit5 et un troisième). Ce modèle n'est pas le seul possible (cf. question iii en annexe). Sur chaque critère, la frontière peut être n'importe où entre deux valeurs voisines des données sans changer les affectations. On l'a placée au milieu de cet intervalle pour qu'elle soit le plus loin possible des données.

Protocole de test

Pour voir si le programme apprend bien, on part d'un modèle connu et on regarde si on le retrouve :

1. On tire au hasard un modèle MR-Sort de référence $\mathcal{M}'$: poids, $\lambda \in [0.5, 1]$, frontières ordonnées.
2. On tire $|L|$ objets aléatoires dans $[0, 1]^4$ et on les classe avec $\mathcal{M}'$: c'est le learning set L .
3. On apprend $\mathcal{M}^*$ sur L avec notre programme.
4. On tire 1000 nouveaux objets T , jamais vus, et on les classe avec $\mathcal{M}'$ et $\mathcal{M}^*$. La *qualité* est la proportion d'objets affectés à la même classe.

On répète 20 fois et on fait la moyenne. On teste sur de nouveaux objets parce que sur L le modèle appris est correct à 100 % par construction.

Résultats

p	$ L $	Qualité moyenne sur T	Écart-type	Temps moyen
2	20	88.5 %	5.4 %	0.05 s
2	50	96.4 %	2.5 %	0.18 s
2	100	98.4 %	1.2 %	0.49 s
3	50	91.5 %	4.3 %	0.31 s
3	100	96.6 %	2.0 %	0.74 s

Dans les 100 essais, le learning set est bien classé à 100 %. La qualité sur T augmente avec la taille de L , car avec plus d'exemples, les frontières sont mieux encadrées et $\mathcal{M}^*$ se rapproche de $\mathcal{M}'$. Avec 3 catégories il faut plus d'exemples, puisqu'il y a deux frontières à apprendre.

Annexe : questions préliminaires (admission d'étudiants)

Question i

Somme des poids des cours validés (≥ 12), avec $w = (0.25, 0.15, 0.2, 0.4)$, $\lambda = 0.66$: Alice $0.15 + 0.2 + 0.4 = 0.75$ ($\mathcal{A}$), Bob $0.25 + 0.15 + 0.2 = 0.60$ ($\mathcal{R}$), Charlie $0.15 + 0.4 = 0.55$ ($\mathcal{R}$), David 1 ($\mathcal{A}$). Bob ne peut pas améliorer sa situation. Avec 12 en physique, ce cours est déjà validé et une meilleure note n'y change rien. Seule une meilleure note en histoire (10) pourrait le faire admettre.

Question ii

Il faut que n'importe quels 3 cours atteignent λ et que n'importe quels 2 cours restent en dessous. Le plus simple est de donner le même poids aux 4 cours, $w_M = w_P = w_L = w_H = \frac{1}{4}$, avec $\lambda \in]\frac{1}{2}, \frac{3}{4}]$, par exemple $\lambda = 0.75$. Alors 3 cours pèsent $\frac{3}{4} \geq \lambda$ et 2 cours pèsent $\frac{1}{2} < \lambda$. Ce n'est pas la seule solution, par exemple $w = (0.3, 0.25, 0.25, 0.2)$ avec $\lambda = 0.6$ convient aussi.

Question iii

Coalitions gagnantes (somme des poids $\geq \lambda$) dans les deux cas :

Coalition	$w = (0.49, 0.49, 0.02)$, $\lambda = 0.5$	$w = (\frac{1}{3}, \frac{1}{3}, \frac{1}{3})$, $\lambda = 0.5$
$\{1\}, \{2\}, \{3\}$	0.49, 0.49, 0.02 : perdantes	0.33 : perdantes
$\{1, 2\}$	0.98 : gagnante	0.67 : gagnante
$\{1, 3\}, \{2, 3\}$	0.51 : gagnantes	0.67 : gagnantes
$\{1, 2, 3\}$	1 : gagnante	1 : gagnante

Dans les deux cas, un étudiant est admis si et seulement s'il valide *au moins 2 cours sur 3*, donc les deux jeux de paramètres définissent la même règle.

On en conclut que (w, λ) n'est pas unique. Ce qui compte, ce sont les coalitions gagnantes. Les poids ne donnent pas directement l'importance d'un cours (le cours 3 ne pèse que 0.02 mais il est aussi décisif que les autres). Pour juger un modèle appris, il faut donc regarder ses décisions plutôt que ses paramètres.

Question iv

b étant connu, on connaît pour chaque objet x l'ensemble des critères qu'il valide, $C(x) = \{i \in \mathcal{N} : x_i \geq b_i\}$, et son score $s(x) = \sum_{i \in C(x)} w_i$ est linéaire en w . Les variables sont les $w_i \geq 0$, $\lambda \in [0, 1]$ et une marge α .

$$\begin{aligned}
 & \max \alpha \\
 & \text{s.c. } s(x) \geq \lambda + \alpha & \forall x \in A^* \\
 & \quad s(x) \leq \lambda - \alpha & \forall x \in R^* \\
 & \quad \sum_{i \in \mathcal{N}} w_i = 1
 \end{aligned}$$

α est le plus petit écart entre un score et λ . En le maximisant, on éloigne le plus possible les acceptés et les rejetés du seuil. Si $\alpha^* > 0$, tous les objets sont bien classés, sinon aucun modèle ne colle parfaitement aux données.

